

Project 2: CPU Scheduling (Deadline: 02/28)

Introduction

This project should be programmed entirely in C and must compile and run on the standard Linux environments, e.g., Ubuntu. Copying & pasting code from other teams or from the web is considered cheating and results in an automatic F.

In this assignment, you are asked to implement a multi-threaded program that will allow us to measure the performance (i.e., Throughput, Turnaround time, and Waiting time in Ready Queue) of the four basic CPU scheduling algorithms, namely, FCFS, SJF, Priority (PR), and Round-Robin (RR). Your program will be emulating/simulating the processes whose priority, sequence of CPU burst time(ms) and I/O burst time(ms) will be given in an input file.

Assume that all scheduling algorithms except RR will be non-preemptive, and all scheduling algorithms except PR will ignore process priorities (i.e., all processes have the same priority in FCFS, SJF and RR). Also assume that there is only one IO device and all IO requests will be served using that device in a FIFO manner.

Objectives

- Learn and practice CPU scheduling algorithms by implementing them
- Learn and practice process/thread synchronization mechanisms by implementing them
- Practice system calls and library functions

Deliverables

- Bunch of .c files. [2 points for each algorithm you implement]
- You must create a **Makefile** to compile your source. [0.5 points]
- Each should compile successfully when compiled with the **-Wall** and **-Werror** flags.
- Each should (hopefully) pass tests we supply.
- Include a single README.md for all the files describing the implementation. [0.5 points]
- Please submit your deliverables using Canvas before 11:59pm on the due date.
- Please include all files (including Makefiles, README.md, and source code) necessary for a successful build as a zip file.
- The name of your submission should be *cs460-project2-TeamID.zip*.

1. Input and Output Format

Your program will take the name of the scheduling algorithm, related parameters (if any), and an input file name from command line. Here is how your program should be executed:

```
prog -alg [FIFO|SJF|PR|RR] [-quantum [integer(ms)]] -input [file name]
```

The output of your program will be as follows:

```
Input File Name           : file name  
CPU Scheduling alg       : FIFO|SJF|PR|RR (quantum)  
Throughput               : ...  
Avg. Turnaround time     : ...  
Avg. Waiting time in ready queue: ...
```

The input file is formatted such that each line starts with *proc*, *sleep*, *stop* keywords.

Following *proc*, there will be a sequence of integer numbers: the first one represents the priority (1: lowest, ..., 5: normal, ..., 10: highest). The second number shows the number of CPU/IO bursts in the execution of a process. The remaining integers represents CPU burst and I/O burst times (ms) in an alternating manner. The last number will be the last CPU burst time after which that process exits. Following *sleep*, there will be an integer number representing the time (ms) after which there will be another process.

So one of the threads in your program (e.g., `FileRead_thread()`) would be responsible for processing this file as follows. As long as it reads *proc*, it will create a new process and put it in a ready queue (clearly this process is not an actual one, it will be just a simple data structure (similar to PCB) that contains the given priority and the sequence of CPU burst and I/O burst times, and other fields). When this thread reads *sleep X*, it will sleep X ms and then try to read new processes from the file. Upon reading *stop*, this thread will quit.

Here is a sample input file (copy/paste this to create an input.txt):

```
proc 1 7 10 20 10 50 20 40 10  
proc 1 5 50 10 30 20 40  
sleep 50  
proc 2 3 20 50 20  
proc 3 3 20 50 20  
sleep 50  
proc 2 3 20 50 20  
stop
```

Scheduling

In this program you need at least three threads:

As described above, one thread (say **FileRead_thread**) will read the above file, create a process (not a real process just a data structure representing the characteristic of the process that will be emulated/simulated), and puts it in a ready queue.

CPU scheduler thread will check ready queue; if there is a process, it will pick one according to the scheduling algorithm from ready queue and hold CPU resource for the given CPU burst time (or for quantum time if the scheduling algorithm is RR). That means CPU thread will simply sleep for the given CPU burst time to emulate/simulate the process. Then it will release CPU resource and put this process into IO queue (or ready queue if RR is used) or just terminate if this is the last CPU burst. Then CPU scheduler thread will check ready queue again and repeat the same for the next process.

I/O system thread will check IO queue; if there is a process, it will sleep for the given IO burst time to emulate/simulate the usage of IO device. It then puts this process back into ready queue. Finally it will check IO queue and repeat the same ...

Basically you will have at least the above mentioned three threads and the main one. These threads need to be synchronized as they cooperate to collect data for performance measures and share data through ready and IO queues. Main thread will wait until the file reading thread is done and the ready queue and IO queue are empty, then it will print the performance evaluation results and terminate the program... You are asked to design and implement this program in C using pthread (no Java).

Also you can use any data structures in different parts of your program and share them along with new variables; but when it comes to maintaining the list of processes in ready queue or IO queue, we would like you to use **double linked list**, as this is the case in many practical OS.

Test cases

Please check your program with input.txt.

You can then compare your outputs with the ones provided in output.txt. As I mentioned you might be getting different numbers even when you run the same program again! That is fine!

Grading

Your submission will be graded by compiling and running it.

- Please make sure your source code can compile. Absolutely no credit if it does not compile.
- Please make sure your programs run on standard Linux servers.
- Please don't include the binary files. Do a make clean before submission.
- Please don't leave out any files!

- Please don't modify any files you don't need to!
- Please don't send us the meta-information from your revision control system!